임베드 채팅 위젯 연동 가이드

연동 레퍼런스

host 페이지에서 위젯을 호출하는 방법과 옵션 안내

2026년 10월 1일

연동 레퍼런스

목차

I. 개요 및 시작

1 개요 4

2 빠른 시작 4

II. 옵션 및 동작

3 init() 옵션 6

4 동작 범위 7

5 페이지 콘텐츠 커스터마이징 8

III. 적용 및 점검

6 적용 방식 제안 11

7 연동 전 체크리스트 12

8 문제 해결 12

I. 개요 및 시작

1. 개요
2. 빠른 시작

개요 및 시작

01 개요

Intercom, Zendesk와 같은 방식의 **플로팅 채팅 위젯**입니다. host 페이지에서 `embed.js`를 불러온 뒤 `window.MazeloneChatWidget.init()`을 한 번 호출하면, 버튼 렌더링·iframe 생성·통신까지 위젯 스크립트가 처리합니다.

연동 흐름

STEP 01
embed.js 로드

→

STEP 02
init() 1회 호출

이후 플로팅 버튼 표시, 패널 오픈, 채팅 iframe 로드는 위젯 스크립트가 자동으로 처리합니다.

주요 특징

- 화면 모서리에 플로팅 버튼이 표시되고, 클릭하면 같은 쪽에서 리사이즈 가능한 패널이 열려 채팅 UI(iframe)가 나타납니다.
- 위젯 UI는 Shadow DOM으로 격리되어 host 페이지의 CSS와 서로 영향을 주지 않습니다.
- 사용자의 질문에 현재 페이지 내용을 참고할 수 있도록, 위젯은 페이지 콘텐츠를 채팅 앱에 전달합니다. 전달 범위는 `init()` 옵션으로 조정할 수 있습니다 (5항).

02 빠른 시작

옵션 없이 호출하면 기본값으로 동작합니다. `</body>` 직전에 넣거나 `defer` 속성을 사용해 주세요.

HTML

```
<script src="/path/to/embed.js" defer></script>
<script>
  window.addEventListener('DOMContentLoaded', () => {
    window.MazeloneChatWidget.init();
  });
</script>
```

`embed.js`는 파일로 전달드립니다. host 사이트의 정적 파일 경로에 배치한 뒤, `src`를 그 경로로 바꿔 주세요 (`/path/to/embed.js`는 예시입니다). `init()`은 `document.body`에 위젯을 붙이므로 DOM이 준비된 뒤에 호출해야 합니다.

II. 옵션 및 동작

3. init() 옵션
4. 동작 범위
5. 페이지 콘텐츠 커스터마이징

옵션 및 동작

03 init() 옵션

모든 옵션은 선택 사항이며, 생략하면 기본값이 적용됩니다.

옵션 명세

옵션	타입	기본값	설명
position	`'bottom-left' | 'bottom-right'`	`'bottom-right'`	토글 버튼 위치. 패널은 버튼과 같은 쪽에서 열립니다. 기존 플로팅 UI와 겹치면 반대쪽으로 조정합니다.
offset	`{ bottom?: number; side?: number }`	`{ bottom: 20, side: 20 }`	토글 버튼 여백(px) 미세조정.
zIndex	`number`	`2147483647`	위젯(토글 버튼 + 패널)의 z-index. 항상 최상단이어야 하는 오버레이가 있으면 그 아래 값으로 지정합니다.
suggestionPrompts	`string[]`	`[]`	위젯 첫 오픈 시 보여줄 예시 질문 목록. `init()` 호출 시점에 고정되며, 이후 페이지 이동이나 DOM 변경으로 갱신되지 않습니다.
getPageContext	`() => { content: string; contentType: string }`	`document.body.innerHTML` 전체 (`text/html`)	채팅 컨텍스트로 전달할 페이지 콘텐츠를 만드는 함수 (5항).
getSelectionContext	`() => { content: string; contentType: string }`	선택된 텍스트 그대로 (`text/plain`)	사용자가 드래그로 선택한 내용을 가공하는 함수 (5항).

사용 예시

JAVASCRIPT

```
window.MazeloneChatWidget.init({
  position: 'bottom-left',
  offset: { bottom: 80, side: 24 },
  zIndex: 9000,
  suggestionPrompts: ['이 페이지를 요약해 주세요', '문의는 어디로 하나요?'],
});
```

옵션 및 동작

04 스크립트가 하는 일 — 동작 범위

host 페이지에 영향을 주는 동작을 아래에 정리했습니다.

동작 항목

구분	내용
DOM 추가	컨테이너 `<div>` 1개를 `<body>`의 마지막 자식으로 추가하고 Shadow DOM으로 격리합니다. 토글 버튼과 패널은 `position: fixed` 입니다.
표시	• 토글 버튼은 기본 우하단이며, `position`과 `offset`으로 위치를 바꿉니다. • 패널은 화면 높이 전체를 차지하고 기본 너비는 400px입니다. 사용자가 경계를 드래그해 320~800px(화면 너비의 90% 이내)로 조절할 수 있으며, 페이지를 새로 불러오면 기본 너비로 돌아갑니다. • z-index 기본값은 브라우저 지원 최댓값(`2147483647`)이라 다른 요소보다 위에 렌더링됩니다. 항상 최상단이어야 하는 오버레이(안내 팝업, 약관 동의창 등)가 있으면 `zIndex` 옵션을 지정합니다.
iframe 로드	iframe은 별도 도메인의 채팅 앱(`navia.krs.co.kr/bridge-chat`)을 가리키며, **패널을 처음 열 때** 로드합니다. 로드 전에 서버 응답 여부를 먼저 확인해, 연결되지 않거나 8초 안에 준비되지 않으면 오류 화면과 "다시 시도" 버튼을 보여줍니다.
컨텍스트 전송	• 기본값으로 `document.body.innerHTML` 전체를 채팅 컨텍스트로 전달합니다. 채팅 앱이 요청할 때마다 `getPageContext`를 호출하므로, 호출 시점의 현재 페이지 내용이 전달됩니다. • 사용자가 텍스트를 드래그해 선택하면(마우스 버튼을 놓는 시점) 선택한 내용을 별도로 전달합니다. • 특정 영역을 제외하려면 5항을 참고해 주세요.
통신	host와 iframe은 `postMessage`로 통신하며, 수신 시 origin을 검증해 허용된 origin의 메시지만 처리합니다. 송신 시에도 대상 origin을 채팅 앱으로 고정합니다.
전역 이벤트 리스너	아래 3개를 등록합니다. 페이지의 기존 이벤트 처리에는 영향을 주지 않습니다. • `window`의 `message` : 채팅 앱(iframe)과의 통신 수신 • `window`의 `resize` : 창 크기가 바뀔 때 패널 너비 보정 • `document`의 `mouseup` : 드래그로 선택한 텍스트 감지
중복 호출	`init()`은 페이지당 한 번만 동작합니다. 다시 호출하면 콘솔에 경고를 남기고 무시합니다.

옵션 및 동작

05 페이지 콘텐츠 커스터마이징

기본값으로는 4항에서 안내한 대로 `document.body.innerHTML`, 즉 페이지의 HTML 전체가 채팅 컨텍스트로 전달됩니다. 로그인 정보나 개인정보처럼 전달되면 안 되는 영역이 있다면, `init()`에 `getPageContext` 함수를 전달해 직접 제외해 주세요. 페이지 구조와 민감정보 영역은 host 사이트에서 가장 정확히 알 수 있기 때문에, 위젯이 임의로 필터링하지 않고 host가 작성한 함수를 그대로 따릅니다.

getPageContext / getSelectionContext

옵션	호출 시점	기본 동작	필요한 경우
`getPageContext`	채팅 앱이 컨텍스트를 요청할 때마다	`document.body.innerHTML` 전체를 `text/html` 로 전달	민감정보 영역 제외 (선택)
`getSelectionContext`	사용자가 드래그 선택을 마칠 때(`mouseup`)마다	선택된 텍스트를 앞뒤 공백 제거 후 `text/plain` 으로 전달	선택 텍스트 가공 (선택)

반환 형식

두 함수 모두 `{ content: string, contentType: string }` 형태의 객체를 반환해야 합니다. `contentType`은 `'text/html'`, `'text/plain'` 같은 MIME 타입 문자열입니다.

예시 1. 특정 영역 제외

HTML

```
<script>
  window.addEventListener('DOMContentLoaded', () => {
    window.MazeloneChatWidget.init({
      getPageContext: () => {
        // id="sensitive-area"로 표시된 영역을 제외하고 전달
        const clone = document.body.cloneNode(true);
        clone.querySelectorAll('#sensitive-area').forEach((el) => el.remove());
        return { content: clone.innerHTML, contentType: 'text/html' };
      },
    });
  });
</script>
```

예시 2. 선택 텍스트 가공

JAVASCRIPT

```
window.MazeloneChatWidget.init({
  getSelectionContext: () => {
    const text = window.getSelection()?.toString().trim() ?? '';
    // 너무 긴 선택은 앞부분 1,000자만 전달
    return { content: text.slice(0, 1000), contentType: 'text/plain' };
  },
});
```

두 함수는 한 번의 `init()` 호출에 함께 전달할 수 있습니다.

주의 오류가 나면 해당 전송은 건너뜁니다. 함수가 예외를 던지거나 `{ content: string, contentType: string }` 형식이 아닌 값을 반환하면, 위젯은 콘솔에 경고를 남기고 **그 메시지를 보내지 않습니다.** `getPageContext`가 실패했다고 해서 기본값(페이지 전체)으로 대체해 전송하지는 않으므로, 제외하려던 영역이 실수로 전달되지는 않습니다. `getSelectionContext`가 빈 `content`를 반환하는 경우에도 전송하지 않습니다.

III. 적용 및 점검

6. 적용 방식 제안
7. 연동 전 체크리스트
8. 문제 해결

적용 및 점검

06 적용 방식 제안 — 공통 레이아웃에 1회 삽입

위젯은 어떤 페이지에서도 동일하게 동작하므로, 페이지마다 개별로 스크립트를 삽입하실 필요가 없습니다. 사이트의 모든 페이지가 공통으로 거치는 템플릿(헤더/푸터 include, 공통 레이아웃 파일 등)이 있다면, 그 한 곳에만 스크립트 태그를 추가하는 방식을 제안드립니다.

홈페이지·서브페이지·영문(`/eng/`) 페이지의 소스를 비교한 결과, `/Common/Com_JS/common.js`와 `/Common/Com_JS/Common_Default.js` 두 파일이 세 페이지 모두에 동일하게 포함되어 있었습니다. 이 두 파일이 `<script>`로 include되는 템플릿(마스터 페이지)에 위젯 스크립트를 함께 추가하면, 별도 작업 없이 전체 페이지에 일괄 적용될 것으로 예상합니다.

삽입 방식 비교

방식	설명	비고
A. 공통 레이아웃 삽입	모든 페이지가 공유하는 템플릿 파일 1곳에 스크립트 태그 추가	**권장.** 배포 1회로 전체 페이지에 적용됩니다.
B. 페이지별 개별 삽입	페이지 템플릿마다 각각 스크립트 추가	권장하지 않음. 신규 페이지에서 누락될 수 있고 유지보수 부담이 큽니다.

삽입 예시

공통 템플릿 (COMMON.JS 가 INCLUDE되는 위치)

```
<script src="/path/to/embed.js" defer></script>
<script>
  window.addEventListener('DOMContentLoaded', () => {
    window.MazeloneChatWidget.init({
      // 필요한 옵션만 지정 (생략 가능)
    });
  });
</script>
```

- 페이지를 이동할 때마다 위젯이 새로 만들어지므로, 패널 열림 상태와 조절한 너비는 초기화됩니다.
- `suggestionPrompts`는 페이지마다 다른 값을 줄 수 있습니다. 공통 템플릿에서 현재 페이지에 맞는 값을 내려주면 됩니다.
- `/Common/Com_Popup/` 하위의 팝업 전용 페이지(예: 에러 안내 페이지)는 위 두 파일을 포함한 어떤 스크립트도 로드하지 않는 별도의 경량 템플릿을 사용하고 있었습니다. 위젯이 필요 없는 영역이므로 통합 범위에서 자연스럽게 제외됩니다. 이 외에 `common.js`가 포함되지 않는 별도 템플릿을 쓰는 페이지 그룹이 있다면 알려 주세요.

사이트 구조나 운영 방식상 더 적합한 방법이 있다면 언제든 제안해 주세요. 협의해서 조율하겠습니다.

적용 및 점검

07 연동 전 체크리스트

운영 환경에 적용하기 전에 아래 항목을 확인해 주세요.

1. **CSP (Content-Security-Policy).** 정책을 운영 중이라면 `frame-src`에 채팅 앱 도메인(`navia.krs.co.kr`)을 추가해 주세요. 위젯은 iframe을 띄우기 전에 채팅 서버 응답 여부를 `fetch`로 확인하므로, `connect-src`(또는 `default-src`)에도 같은 도메인이 허용되어야 합니다.
2. **기존 플로팅 UI와의 위치.** 상담원 연결, 챗봇, 맨 위로 가기 버튼이 화면 모서리에 이미 있다면 `position`과 `offset`으로 겹치지 않게 조정해 주세요.
3. **항상 최상단이어야 하는 오버레이.** 페이지 전체를 덮는 안내 팝업이나 약관 동의창이 있다면, 위젯이 그 아래로 내려가도록 `zIndex`를 오버레이보다 낮게 지정해 주세요.
4. **민감정보 영역.** 채팅 컨텍스트로 전달되면 안 되는 영역이 있다면 `getPageContext`로 제외해 주세요 (5항).

08 문제 해결

증상	확인할 내용
버튼이 보이지 않는다	`embed.js`가 로드되었는지(네트워크 탭에서 경로 오류나 404 여부), `init()`이 호출되었는지, 다른 요소가 버튼을 가리고 있지 않은지 확인합니다. 콘솔에 오류가 있는지도 확인해 주세요.
"채팅 서버에 연결할 수 없습니다" 화면	채팅 서버에 연결되지 않은 경우입니다. 네트워크 상태와 CSP `connect-src`·`frame-src`에 채팅 앱 도메인이 허용되어 있는지 확인한 뒤 "다시 시도"를 눌러 주세요.
"채팅을 불러올 수 없습니다" 화면	서버에는 연결되었지만 8초 안에 채팅 UI가 준비되지 않은 경우입니다. 잠시 후 "다시 시도"를 눌러 주세요.
다른 플로팅 UI와 겹친다	`position`을 반대쪽으로 바꾸거나 `offset`으로 여백을 조정합니다.
팝업·오버레이가 위젯에 가려진다	`zIndex`를 오버레이보다 낮은 값으로 지정합니다.
콘솔: "init()이 이미 호출되었습니다"	`init()`이 한 페이지에서 두 번 이상 호출된 경우입니다. 호출 위치가 중복되지 않았는지 확인합니다. 동작에는 영향이 없습니다.
콘솔: "getPageContext 처리 중 에러가 발생해…"	함수가 예외를 던졌거나 `{ content: string, contentType: string }` 형식이 아닌 값을 반환했습니다. 이 경우 해당 컨텍스트는 전송되지 않습니다. `getSelectionContext`도 같은 방식으로 확인합니다.

연동 중 궁금한 점이 있으면 언제든 문의해 주세요.